CONVERTLY — Master Build Prompt

HOW TO USE THIS PROMPT

Paste this entire document as the system/context prompt to your AI coding agent at the start of the project. Refer back to specific sections ("see §7.2 Upload Dropzone") in follow-up prompts as you build feature-by-feature. Do not ask for the whole app in one shot — use §15 as your build sequence.

---

1. ROLE & OBJECTIVE

You are a senior full-stack product engineer and UI/UX designer. Your job is to design and build **Convertly**, a zero-login, privacy-first, ultra-fast online file converter that feels as premium and considered as Claude.ai, Linear, Raycast, Arc, and Notion. Every pixel, animation, and API response should feel intentional. Favor restraint over decoration — the brief blends five visual styles, but the execution must feel calm, not maximalist.

Before writing code, propose a folder/architecture plan and confirm it against §4 and §15. When a requirement is ambiguous, make the most premium, most performant choice and state your assumption briefly — don't stall on it.

---

2. PROJECT OVERVIEW

Name: Convertly (working name)

Vision: The fastest, lightest, most beautiful online file converter — zero login, instant conversion, premium feel.

Non-negotiable core principles:

- No login, no signup, ever
- Privacy-first (no permanent file storage)
- Ultra fast (perceived + actual)
- Mobile-first responsive design

- Smooth, GPU-accelerated animation only
- WCAG AA accessible
- SEO-friendly (SSR/SSG where possible)
- Dark + light theme
- Total load time under 2 seconds

Primary inspiration: Claude AI, Apple HIG, Linear.app, Raycast, Notion, Arc Browser.

---

3. DESIGN SYSTEM

3.1 Visual Language — Where Each Style Applies

Do not blend all styles everywhere. Apply them surgically per this map:

Style	**Applies To**
Swiss Design	Overall grid, layout rhythm, spacing (8pt grid), alignment
Minimalism	Typography, navigation, iconography
Glassmorphism	Navbar, modals/dialogs, upload panel background
Claymorphism	Primary buttons, file cards, tool cards
Neomorphism	Input fields only (subtle, low-contrast)
Liquid Glass	Conversion progress bar, loading/processing animations
Soft 3D	Hero centerpiece icon, empty-state illustrations

Everything else stays flat and minimal. If a component isn't listed above, default to minimalism.

3.2 Color Tokens

```
:root {

  --color-primary: #004741;      /* Cyprus */
```

```
  --color-secondary: #F0EDE4;    /* Sand */

  --color-accent: #FFFFFF;

  --color-success: #28C76F;

  --color-warning: #FFB547;

  --color-danger: #EA5455;

  --color-bg: #F8F8F6;

  --color-bg-dark: #111111;

}
```

Define matching dark-mode tokens (e.g. `--color-bg-dark` surfaces, elevated cards at `#1A1A1A`, text at `#F0EDE4`) so contrast stays WCAG AA in both themes. Implement via Tailwind CSS variables + `class="dark"` strategy, not `prefers-color-scheme` alone — give users an explicit toggle.

3.3 Typography

- **Primary (display only):** Kanesty — confirmed real typeface: bold, rounded, high-x-height geometric sans. Strong for the wordmark, hero headline, and large H1/H2 moments, but too heavy and loud at body-text or small UI sizes. Load via `next/font` or self-hosted `@font-face`; restrict its use to display-scale headings and the logo.
- **Body / UI text:** Inter — use for paragraphs, nav labels, buttons, form fields, and anything below ~24px. Keeps the interface calm and legible while Kanesty carries brand personality only where it counts.
- **Fallback stack:** Inter → SF Pro Display → Helvetica → system-ui
- Type scale: Swiss-style modular scale (e.g. 1.25 ratio), generous line-height (1.5+ body, 1.1–1.2 headings)

3.4 Layout & Grid

- 8pt spacing system throughout
- 12-column responsive grid, max content width ~1440px, fluid beyond that for ultrawide
- No horizontal scroll at any breakpoint (test 320px → 3440px)

3.5 Motion Principles

- GPU-accelerated properties only: `transform`, `opacity` (never animate `width`/`height`/`top`/`left` directly)

- Target 60fps, zero cumulative layout shift
- Respect `prefers-reduced-motion`: provide a reduced-motion variant for every animation (crossfade instead of spring/bounce)
- Easing: custom cubic-bezier curves for an "elastic but controlled" feel — avoid default linear/ease

---

4. TECH STACK

Frontend

- Next.js (App Router) + React + TypeScript
- Tailwind CSS (design tokens as CSS variables, see §3.2)
- Framer Motion (primary animation library)
- GSAP — only for complex timeline sequences Framer Motion can't handle cleanly (e.g. the liquid progress bar)

Backend

- FastAPI, Python 3.12+
- Redis + a task queue (`arq`, RQ, or Celery) for job handling — see §8.2 for why this replaced ad-hoc background tasks

Conversion Engines

- FFmpeg (video/audio)
- LibreOffice headless (office docs)
- ImageMagick (images)
- Ghostscript (PostScript/PDF)
- Poppler (PDF utilities)
- Pandoc (document format conversion)

Infra

- Cloudflare (CDN + WAF + Brotli)
- Redis (job queue + live progress state — see §8.2)
- Docker (containerizes the app and a pre-warmed, horizontally-scalable worker pool — not a container per job; see §8.2)
- Railway or Render for initial hosting; design stateless so migration to AWS/GCP later is trivial

---

5. INFORMATION ARCHITECTURE & NAVIGATION

Sticky navbar:

- Logo (left)
- Nav items: Image Tools · PDF Tools · Video · Audio · All Tools · About · Contact
- Theme toggle (light/dark)
- Effects: frosted glass background, blur-intensifies on scroll, soft shadow, smooth hover underline/fill, active-page indicator (animated pill or underline)

Pages:

- `/` — Home (hero + featured tools + how-it-works)
- `/tools/image` , `/tools/pdf`, `/tools/video`, `/tools/audio` — category hubs
- `/convert/[from]-to-[to]` — individual conversion tool pages (SEO-critical, e.g. `/convert/jpg-to-png`) — statically generate these for every supported pair
- `/tools` — All Tools directory/search
- `/about`, `/contact`
- `/privacy`, `/terms` (required given the security/legal claims made)

---

6. PAGE-BY-PAGE SPECS

6.1 Hero (Home)

- H1: **"Convert Files in Seconds"**
- Subhead: **"Fast, Secure and Completely Free. No Sign-up Required."**
- Centerpiece: large drag-and-drop zone with floating soft-3D icon, soft ambient glow, animated gradient border, bounce/scale on hover
- Below fold: trust row (no login / auto-delete / no watermark), category shortcuts, popular conversions

6.2 Tool Category Pages

- Grid of claymorphism tool cards (icon, format pair, short label)
- Search/filter bar (neomorphic input)
- Skeleton loaders while cards populate from data

6.3 Individual Conversion Tool Page

This is the money page — must handle full workflow in §8. States: idle → dragging → file selected/validating → uploading → processing → success/download → error.

6.4 About / Contact

Minimal, Swiss-grid layout, no unnecessary animation — content-first.

---

7. CORE COMPONENT SPECS

7.1 Upload Dropzone (Hero centerpiece)

- Large target area, accepts drag-and-drop and click-to-browse
- Real-time file type + size validation with inline error state (danger color, shake micro-animation)
- Shows accepted formats list contextually
- Animated upload progress (liquid/gradient fill, not a plain bar)

7.2 Custom Cursor

- Gradient glow dot with soft trailing/magnetic pull toward interactive elements
- Scales up on hover over buttons/links
- Ripple effect on click
- **Must** be fully disabled on touch devices and respect `prefers-reduced-motion`

7.3 Skeleton Loaders

Use everywhere instead of spinners: tool cards, image previews, download cards, settings panels, upload area placeholder. Shimmer animation, GPU-accelerated (`transform: translateX` sweep on a gradient overlay).

7.4 Conversion Processing Animation

- Liquid loading bar (organic wave fill, not a straight rectangle)
- Gradient slider + floating particles (canvas or lightweight WebGL/CSS, capped particle count for perf)
- Morphing file-icon animation (input format icon morphs into output format icon)
- Smooth animated percentage counter (no frame skipping)
- Elastic "pop" completion effect → transitions into download card

7.5 Buttons & Cards (Claymorphism)

- Soft dual-shadow (light top-left, dark bottom-right), rounded generous corners, subtle press-down animation on click (`scale(0.97)` + shadow inset)

7.6 Inputs (Neomorphism)

- Low-contrast inset shadow, background matches surrounding surface, focus state adds visible outline ring (accessibility requirement overrides pure neomorphic subtlety — focus rings must remain clearly visible)

7.7 Modals / Dialogs (Glassmorphism)

- `backdrop-filter: blur()`, semi-transparent surface, soft border, elevated shadow

7.8 Micro-interactions (global)

Button press feedback, card lift on hover, floating icons (idle float loop), fade-in on scroll (IntersectionObserver-based, not scroll-jank-prone libraries), soft parallax (transform-only), magnetic buttons, ripple on click.

---

8. CONVERSION WORKFLOW

8.1 Frontend Flow

1. Upload file (drag-drop or picker)
2. Client-side detect file type + validate size/MIME before upload
3. Show output format options relevant to detected input type
4. User selects output format → triggers conversion
5. Upload streams to backend with progress events
6. Frontend subscribes to a single SSE stream for job status (see §8.2)
7. Show liquid progress + percentage
8. On completion, reveal download card with elastic animation
9. Auto-delete source + output file after 30 minutes (server-side scheduled cleanup job, independent of whether the user downloaded it)

8.2 Backend Architecture (revised)

Do **not** spin up a new Docker container per conversion job — container cold-starts (often 1–5s+) directly fight the "ultra fast" principle and add unnecessary complexity to progress tracking. Use this instead:

- **Job queue:** Redis + a task queue (`arq`, RQ, or Celery). The API server only enqueues jobs; it never runs conversions itself.
- **Worker pool:** A small number of long-running, pre-warmed worker containers (stateless, horizontally scalable) pull jobs off the queue. Run one shared pool, or 2–3 pools tuned per media type (video/audio workers get more CPU/memory than image/PDF workers).
- **Sandboxing per job, without container-per-job cost:** each worker runs the actual conversion binary (ffmpeg/ImageMagick/etc.) as a subprocess under a locked-down, non-root user, in a dedicated temp directory that's wiped after the job, with strict `ulimit`/cgroup resource caps and a hard timeout that kills runaway processes. This captures most of the isolation benefit of a fresh container at a fraction of the latency. If the threat model later demands stronger isolation, upgrade the worker pool to gVisor or Firecracker microVMs rather than Docker-per-job — same goal, far lower overhead.
- **Progress reporting:** workers write progress (parsed from ffmpeg's `-progress` output, or checkpoint percentages for other tools) into a Redis hash keyed by `job_id`. The API server subscribes to Redis and forwards updates to the client over one SSE connection. This decouples the client-facing transport from the compute layer, so workers can scale, restart, or fail over without the browser's connection caring which worker did the work.

8.3 API Surface (FastAPI)

- `POST /api/upload` → returns `job_id`, does MIME + size + (async) virus-scan validation
- `POST /api/convert/{job_id}` → body: `{ target_format }`, enqueues the job
- `GET /api/status/{job_id}` (SSE) → streams progress % and state from Redis
- `GET /api/download/{job_id}` → signed, short-lived, randomized-filename download URL
- `DELETE` handled automatically via TTL/cron, not user-triggered

Net effect: fast job pickup (no container cold start), simple horizontal scaling (add worker replicas), and isolation that's "good enough" for untrusted file conversion — without the operational cost of orchestrating a container per request.

---

9. BACKEND & SECURITY REQUIREMENTS

- HTTPS only (HSTS enabled)
- Files auto-delete after 30 minutes — hard TTL, enforced server-side regardless of client state
- No permanent storage; storage layer should be treated as ephemeral (e.g. object storage bucket with lifecycle rule as a backstop)
- Strict file size limits (define per file type, e.g. video larger cap than image)

- MIME type validation via content sniffing, not just file extension
- Virus scanning (e.g. ClamAV) before any conversion runs
- Rate limiting per IP/session (e.g. token bucket at the edge via Cloudflare + app-level limits)
- Randomized filenames (UUID) — never trust or reuse user-supplied filenames on disk
- CSRF protection on any state-changing request; XSS protection via strict output encoding
- Content-Security-Policy header locked down (no inline scripts, allow-list CDN origins only)
- No third-party analytics/trackers that read file contents; if analytics are added later, they must be privacy-respecting (e.g. Plausible) and file-metadata-blind

---

10. PERFORMANCE BUDGET

- Lighthouse score 98+ across Performance/Accessibility/Best Practices/SEO
- First Contentful Paint < 1.2s
- Total interactive load < 2s
- Lazy-load below-the-fold images/components
- Route-level code splitting (Next.js does this by default — don't undo it with premature bundling)
- Image optimization via `next/image` (AVIF/WebP with fallback)
- Brotli compression at the edge (Cloudflare)
- CDN-ready static assets, cache-control headers tuned per asset type
- PWA: installable, offline shell (though conversion itself requires network), app manifest + icons

---

11. ACCESSIBILITY REQUIREMENTS

- WCAG AA contrast minimums for every color combination in both themes
- Full keyboard navigation (tab order, skip-to-content link, no keyboard traps in modals)
- Screen reader support: semantic HTML, ARIA live regions for upload/conversion progress announcements
- Visible focus states on every interactive element (even where neomorphism wants subtlety — accessibility wins)
- High-contrast mode support
- `prefers-reduced-motion` fully honored — disable custom cursor, floating icons, parallax, and bounce/elastic effects in favor of simple fades

---

12. SEO REQUIREMENTS

- SSR/SSG for all tool and conversion pages (Next.js App Router with static generation per format pair)
- Unique title/meta description per conversion page (e.g. "Convert JPG to PNG Free — Convertly")
- Structured data (Schema.org `SoftwareApplication` / `HowTo` where relevant)
- Sitemap.xml + robots.txt
- Semantic heading hierarchy, descriptive alt text, canonical URLs

---

13. DEVICE SUPPORT

Fully responsive, zero horizontal scroll, tested across: mobile, tablet, laptop, desktop, ultrawide monitors, foldable devices (handle the fold/hinge viewport gracefully — avoid critical UI in the crease region on foldables where detectable via viewport segments API).

---

14. DEPLOYMENT

- Docker containers for both the Next.js app and FastAPI conversion service (separate images; conversion workers scale independently)
- Cloudflare in front for CDN, caching, WAF, and Brotli
- Initial hosting: Railway or Render
- Environment-based config (no secrets in code); structure for easy migration to a larger cloud provider as volume grows

---

15. RECOMMENDED BUILD ORDER (use this to sequence your prompts to the AI agent)

Phase 1 — Foundation

1. Next.js + TypeScript + Tailwind scaffold, design tokens from §3.2, dark/light theme toggle
2. Global layout: navbar (§7 nav spec) + footer, responsive shell, no horizontal scroll baseline

Phase 2 — Core Conversion Experience 3. Upload dropzone component (§7.1) with client-side validation 4. FastAPI backend skeleton: upload endpoint, job model, Redis queue + one worker, TTL cleanup job 5. Wire one real conversion pipeline end-to-end (pick one easy pair, e.g. PNG↔JPG) before generalizing 6. Progress/status streaming (SSE) + liquid progress animation (§7.4) 7. Download card + auto-delete confirmation

Phase 3 — Breadth 8. Generalize conversion pipeline to all format families (image/PDF/video/audio) via the engines in §4 9. Generate all `/convert/[from]-to-[to]` static pages + category hub pages

Phase 4 — Polish 10. Custom cursor, skeleton loaders everywhere, micro-interactions pass (§7.8) 11. Accessibility audit (§11) and Lighthouse pass (§10) 12. PWA manifest + offline shell

Phase 5 — Hardening & Launch 13. Security pass: rate limiting, virus scanning, CSP, headers (§9) 14. SEO pass (§12), sitemap, structured data 15. Deploy pipeline (§14)

---

16. FUTURE ROADMAP (post-launch, do not build in v1)

- Batch conversion
- AI-powered OCR
- AI document summarizer
- Background remover
- AI image upscaler
- Browser extension
- Android app
- Public API
- Desktop app

---

17. NON-NEGOTIABLE CONSTRAINTS (recap)

- No login/signup anywhere in the flow
- No file is ever stored beyond 30 minutes
- No animation may cause layout shift or drop below 60fps
- No visual style from §3.1 may be used outside its assigned component
- Every interactive element must be keyboard- and screen-reader-accessible
- Every animated effect needs a reduced-motion fallback